

Arctos CAN Interface

How the arm's motors and CAN bus actually work

Consolidated from project debugging logs, source inspection and bench testing

September 2026

Contents

1	About these documents	2
2	Core concepts	2
2.1	CAN bus, in one paragraph	2
2.2	Gearboxes and closed-loop steppers, in one paragraph	2
3	Hardware	3
3.1	Joint, motor and CAN ID map	3
3.2	Motor specifications	3
3.3	CANable 2 adapter and wiring	4
3.4	Bus termination check, power off	4
4	The CAN interface on Ubuntu	4
5	Controller panel configuration	5
6	The MKS CAN protocol	6
6.1	Frame structure and checksum	6
6.2	Command reference	6
6.3	Reading the encoder	7
6.4	Encoder non-linearity	7
7	Motion commands	7
8	Speed and its limits	8
8.1	Measured ceilings, per joint	9
9	Converting encoder counts to joint degrees	9
10	Endstops and homing	10
10.1	Where the sensor actually connects	10
10.2	Enabling the remap	10
10.3	Reading the pins: 0x34	11
10.4	Homing	11
11	Safety	12

12 Troubleshooting	12
12.1 Total silence with a powered driver: the RS485 variant	13
13 The scripts in this package	13
14 ROS 2 package layout	14
15 Corrections, and what this set replaces	14

1 About these documents

This is the reference document: how the bus works, how the protocol is framed, how to turn encoder counts into joint angles, and what has actually been verified against hardware rather than assumed. It is deliberately not a step-by-step procedure.

There are four documents in this set and they do not overlap:

- **Arctos_CAN_Interface** — this one. Background and reference.
- **Nema_17_MKS42D_SOP** — bringing up a NEMA 17 joint on an MKS SERVO42D. Joints A, B and C.
- **Nema_23_MKS57D_SOP** — bringing up a NEMA 23 joint on an MKS SERVO57D. Joint X. Largely unverified; see that document's opening.
- **BASIC_CAN_COMMANDS** — terminal commands only. Setting the interface up, talking to a motor, shutting down to unplug.

This set replaces `arctos_motor_guide`, `sop_nema17`, `sop_nema23`, `arctos_CAN_ROS2_SOP_updated` and `arctos_motor_can_cheat_sheet_updated`. Section 15 lists the things that were *wrong* in those files, so nothing is silently re-derived from a stale copy.

2 Core concepts

2.1 CAN bus, in one paragraph

CAN is a differential two-wire multi-drop bus. Every device shares the same pair, `CAN_H` and `CAN_L`, plus a common ground, and each device is addressed by an ID. Every node must agree on the bitrate; this arm runs at **500 kbit/s**. Because all nodes share one pair, two devices configured with the same CAN ID will collide and neither will behave correctly.

2.2 Gearboxes and closed-loop steppers, in one paragraph

Each joint is a stepper motor driving a reduction gearbox. The reduction multiplies torque and divides both speed and angular error: through a 67.82:1 gearbox, one motor revolution is only about 5.31° at the joint. The MKS controllers are closed-loop — a magnetic encoder on the motor shaft reports true position, so the controller can tell that it has fallen behind. Two consequences matter constantly in practice. The encoder measures the **motor shaft, not the joint output**. And a stalled stepper converts its full phase current into heat, with no motion to carry it away.

3 Hardware

3.1 Joint, motor and CAN ID map

Joint	Role	CAN ID	Motor / driver	Gear ratio
X	Base and waist rotation	0x01	NEMA 23 / SERVO57D	13.5:1
Y	not confirmed	0x02	—	150.0:1
Z	not confirmed	0x03	—	150.0:1
A	Elbow	0x04	NEMA 17 / SERVO42D	48.0:1
B	Wrist rotation	0x05	NEMA 17 / SERVO42D	67.82:1
C	Wrist joint	0x06	NEMA 17 / SERVO42D	67.82:1

Caution: A driver can be found on the wrong CAN ID — fix the panel, not the map

The table above is the *intended* assignment and matches `arctos_controller.yaml`. A controller’s panel can hold a different value. On 2026-09-02 a scan with only Joint B connected returned a single responder at 0x06, which is Joint C’s ID: that driver’s panel had simply been saved wrong. The work done that day was on Joint B regardless of the ID it answered on.

When a scan disagrees with this table, that is evidence the **panel needs reprogramming**, not evidence the map is wrong. Reprogram the driver rather than editing the map, and never drive a joint until the scan and the map agree. Joint A’s 0x04 and its 48:1 ratio are empirically verified; the rest are config values.

Caution: Always confirm a CAN ID against real hardware

Run the read-only scan before trusting any table, including the one above. It sends only the 0x31 query and never enables coils:

```
cd ~/arctos_ws/src/arctos_can_control/arctos_can_control
python3 can_id_scan.py --channel can0 --ids 1-16
```

3.2 Motor specifications

Parameter	NEMA 17 / SERVO42D	NEMA 23 / SERVO57D
Holding torque	24 N · cm	1.8 N · m
Rated current	1.2 A	2.8 A
Shaft diameter	5 mm	6.35 mm
Step angle	1.8°, 200 full steps per rev	1.8°, 200 full steps per rev
Microstepping	16 times, 3200 pulses per rev	16 times, 3200 pulses per rev
Encoder	16384 counts per rev, 14-bit	16384 counts per rev, 14-bit
Joints	A, B, C	X

Caution: Do not confuse motor steps, microstep pulses and encoder counts

Three different resolutions are in play, and mixing them is the single most common arithmetic error in this project.

- **200** full steps per motor revolution, the physical stepping resolution.
- **3200** microstep pulses per motor revolution, the scale that 0xFD position commands are expressed in.
- **16384** encoder counts per motor revolution, the scale that 0x31 reports in.

One microstep pulse is $16384/3200 = 5.12$ encoder counts. Commanded motion and measured motion are therefore never in the same units.

3.3 CANable 2 adapter and wiring

- USB VID:PID 16d0:117e.
- It contains its own CAN controller. Do **not** add an external MCP2515 module in series with it.
- Only three wires are needed between adapter and controller: CAN_H, CAN_L and GND. Do *not* connect the CANable's power lines to the motor controller.

```
Ubuntu host -> USB -> CANable 2 -> CAN_H / CAN_L / GND
              -> MKS controller (SERVO42D or SERVO57D) -> joint motor
```

3.4 Bus termination check, power off

With everything powered off, measure resistance between CAN_H and CAN_L.

Reading	Meaning
About 60 Ω	Correct. Two 120 Ω terminators in parallel
About 120 Ω	Only one terminator present
Open circuit	Wiring is disconnected
Very low	Possible short

4 The CAN interface on Ubuntu

The CANable presents as a serial device, and `slcand` bridges it to a Linux network interface. This has to be rebuilt every time the adapter is plugged in or re-enumerates. The commands themselves are in `BASIC_CAN_COMMANDS`; what follows is why they are what they are.

Caution: The `slcand` bitrate flag is a coded index, not a number

`-s0` is 10k, `-s1` 20k, `-s2` 50k, `-s3` 100k, `-s4` 125k, `-s5` 250k, **`-s6` 500k**, `-s7` 800k, `-s8` 1M. This arm is 500 kbit/s, so `-s6` is correct. Earlier scripts in this project used `-s3` and silently attached at 100 kbit/s, which presents as a completely dead bus.

Caution: On `slcan` the bitrate is set at attach time and `ip` cannot show it

Do not use `ip link set can0 up type can bitrate 500000`. That netlink attribute is for native CAN controller drivers and is not supported on `slcan`. As a result `ip -details link show can0` reports `bitrate 0` and `clock 0` on a perfectly healthy CANable. This is normal and is not a fault to chase. An older version of this documentation told the reader to verify by looking for `bitrate 500000`; that check can never pass on this hardware.

Caution: Kernel CAN error counters are meaningless on `slcan`

The re-started, bus-errors, arbit-lost, error-warn, error-pass and bus-off counters in `ip -details -statistics link show can0` are not populated by the `slcan` driver. They read zero regardless of what the bus is doing. Do not use them as evidence that a bus is healthy, and note that the “total silence with zero bus errors implies no CAN transceiver” heuristic in Section 12.1 needs a native CAN controller to mean anything.

5 Controller panel configuration

Confirm these on the controller’s own display before attempting CAN communication.

Setting	Value
CAN ID	the joint’s ID from the map above
CAN Rate	500
CAN Response, shown as CANRSP	ENABLE
CAN CRC	SUM-8
Working current	at or below the motor’s rated current
Working mode	see the caution below

Save and power-cycle the controller after any change. The CAN ID in particular must be committed, not merely displayed.

Caution: Set the working current to the motor’s rating, not the panel default

A controller in this project was found storing a 3000 mA default against a motor rated 1.2 A, roughly two and a half times over. Check this on every new controller.

Caution: A working-mode mismatch can imitate a communication fault

The ROS 2 driver defines six modes and defaults to `CR_vFOC`, which is 2:

```
CR_OPEN = 0, CR_CLOSE = 1, CR_vFOC = 2,  
SR_OPEN = 3, SR_CLOSE = 4, SR_vFOC = 5
```

If the panel’s mode disagrees with what a motion command assumes, the controller can accept a frame and acknowledge its checksum without rotating the shaft. Confirm the mode before concluding that a “no motion” result is a wiring or CAN problem.

6 The MKS CAN protocol

6.1 Frame structure and checksum

Every frame is [command byte][data bytes][checksum byte], where the checksum is an 8-bit sum that **includes the CAN ID**:

```
def checksum(motor_id, data_bytes):  
    return (motor_id + sum(data_bytes)) & 0xFF
```

Frame, no checksum	CAN ID	Arithmetic	Checksum
31	0x04	0x31+0x04	35
31	0x06	0x31+0x06	37
3A	0x05	0x3A+0x05	3F
F3 01	0x05	0xF3+0x01+0x05	F9
F7	0x06	0xF7+0x06	FD

The checksum differs per joint, so a frame copied from another joint's documentation is silently rejected. BASIC_CAN_COMMANDS has a ready-made table for every ID on this arm.

Caution: Never hand-invent a checksum, and never reuse another joint's frame

Omitted or incorrect checksum bytes were the direct cause of several “the motor ignored my command” failures in this project. Compute it from the formula every time, including for the emergency stop.

6.2 Command reference

Name	Byte	Status
QUERY_MOTOR	0xF1	Liveness check. Answers F1 01.
READ_ENCODER	0x31	Verified. Use this one.
READ_ENCODER carry form	0x30	Superseded. See Section 15.
READ_VELOCITY	0x32	Responds; zero at rest. Layout unconfirmed.
READ_ERROR	0x39	Signed 32-bit deficit. Grows when stalled.
READ_ENABLE_STATE	0x3A	01 enabled, 00 disabled. Verified.
READ_SHAFT_PROTECTION	0x3E	00 clear. Non-zero latches the driver.
READ_IO	0x34	Input and output pin states. Verified. Section 10.
SET_WORKING_MODE	0x82	See the panel caution.
SET_HOME_PARAMS	0x90	Trigger level, direction, speed, EndLimit.
GO_HOME	0x91	Runs onto the limit. Unlocks the shaft if EndLimit=1.
LIMIT_PORT_REMAP	0x9E	Required for 42D endstops here. No read-back.
ENABLE_SHAFT_PROTECTION	0x88	Avoid. False positives, and it latches.
ENABLE_MOTOR	0xF3	01 enable, 00 disable. Verified.
RELATIVE_POSITION	0xF4	Unsafe packing. See the motion section.
ABSOLUTE_POSITION	0xF5	Unsafe. See the motion section.

Name	Byte	Status
SPEED_CONTROL	0xF6	Not exercised.
EMERGENCY_STOP	0xF7	Verified, with checksum.
POSITION_CONTROL	0xFD	Verified. Preferred for all motion.

6.3 Reading the encoder

Use 0x31. The response is [0x31][6 bytes signed position][checksum], where the six bytes are a big-endian signed 48-bit multi-turn count on the 16384-counts-per-revolution scale:

```
raw48 = int.from_bytes(data[1:7], byteorder="big", signed=True)
motor_degrees = raw48 * 360.0 / 16384
joint_degrees = motor_degrees / gear_ratio
```

This is multi-turn and does not wrap at one revolution. It resets to approximately zero when the controller is power-cycled, so it is a relative reference from power-on, not an absolute machine coordinate.

Caution: The encoder measures the motor shaft, not the joint output

Gearbox backlash and any lost motion downstream of the motor are invisible to this encoder. It cannot measure output-side backlash; that needs a dial indicator on the output.

6.4 Encoder non-linearity

The magnetic encoder carries a once-per-revolution sinusoidal error caused by the sensing magnet sitting slightly off-centre on the shaft. Measured on Joint A over 6725 samples: **1.465° peak to peak at the motor shaft**, about 0.407% of a revolution.

Two practical consequences:

- It cancels over any whole number of motor revolutions. Choosing a step size that is an integer number of motor turns makes it vanish — Joint A steps of exactly 6 motor revolutions landed at 100.0% every time, where 90° motor-shaft steps scattered between 98.7 and 101.4%.
- Through a reduction it becomes negligible at the joint: 1.465° through 48:1 is 0.031°.

Repeatability is unaffected. Returning to the same shaft angle repeats to better than one encoder count. Absolute readings carry the cyclic error; relative and repeat positioning do not.

7 Motion commands

Use **POSITION_CONTROL**, 0xFD. It is what the project's own working jog code uses and the only motion command verified safe here.

```
[0xFD]
[direction | ((speed >> 8) & 0x0F)]  direction 0x00 fwd, 0x80 reverse
[speed & 0xFF]                      speed is approximately motor rpm
[accel]
```


[(pulses >> 16) & 0xFF]	ALWAYS-POSITIVE magnitude, on the
[(pulses >> 8) & 0xFF]	3200-pulses-per-motor-rev scale
[pulses & 0xFF]	
[checksum]	

$$\text{pulses} = |\text{joint degrees}| \times \frac{3200 \times \text{gear ratio}}{360}$$

The driver replies with an unsolicited two-frame status sequence: FD 01 for “started”, then FD 02 for “complete” if the commanded distance was actually achieved. **The absence of FD 02 is the reliable signal that a command did not finish.**

Caution: Do not use 0xF5, and be careful with 0xF4

0xF5 packs an absolute target through a plain & 0xFFFFF. On a joint whose raw counter had drifted to roughly −274000 counts, a small intended nudge became an enormous, non-settling, uninterruptible move that required cutting motor power. 0xF4 packs a possibly-negative delta the same way. The 0xFD pattern above, a separate direction byte plus an always-positive magnitude, does not have this failure mode.

Caution: Never retry a stalled command into an obstruction

Re-sending the same command to a joint that has not moved changes nothing except the motor’s temperature. Nine consecutive full-torque retries into a hard stop drove one joint’s internal position error to −124374 counts and cost a cooldown to recover from. If a step under-delivers, stop, or back off and re-attempt once.

Caution: A repeatable stop angle is mechanical, not roughness

Rough gear mesh varies. If a joint stops at the *same* angle to the encoder count on attempt after attempt, and the back-off also delivers nothing, that is a hard mechanical limit. On Joint B this pattern turned out to be a gear seated in the wrong position; once corrected the joint swept the same range with zero unstick cycles. Open the gearbox rather than working at it.

8 Speed and its limits

The *speed* field is very close to motor rpm. Measured on a bare NEMA 17 with no load:

Commanded	Actual	Behaviour
10–225	tracks to within 0.5%	linear
250	98.7%	edge of linear
275–325	96.5% down to 90%	rolling off
350 plus	saturates near 310 rpm	losing steps

8.1 Measured ceilings, per joint

That table is a bare motor on a bench. A joint drags its gearbox with it and the ceiling falls accordingly, so treat the no-load numbers as an upper bound and measure each joint.

Joint	Driver	Gear	Linear to	Saturates
bench, no load	SERVO42D	none	225 rpm	about 310 rpm
B	SERVO42D	67.82:1	150 rpm	about 165 rpm

Joint B was measured on 2026-09-02 over a 48° arc. Commanded 150 tracked at 99.7%; commanded 165, 175, 200, 225 and 300 all produced 159 to 165 rpm. Past saturation, extra commanded speed buys nothing but current and heat. The measurement was repeated at eight times the acceleration to confirm the ramp was not the limiter, which moved the result by only about 4%.

Caution: Over-commanding speed did not lose position on a geared joint

0xFD commands a pulse *count*, not a duration. On Joint B every trial from 150 through 300 commanded delivered its 48° arc to +0.01%: past saturation the move simply takes longer, it does not drop pulses. Silent position loss requires actual slip. Do not carry the no-load “loses steps above 250” result across to a geared joint automatically — verify which behaviour the joint in front of you shows.

Caution: Heat is the real limit on any speed or reliability testing

A ladder of trials run back to back overheats the motor even when no single trial is aggressive. Eleven trials with one-second gaps was enough to have to stop and let a joint cool; fifteen seconds between trials was not. There is no temperature-read opcode on these drivers, so software cannot detect this. Budget total energized time, not just peak current, and stop as soon as you have the answer.

9 Converting encoder counts to joint degrees

$$\text{counts per joint degree} = \frac{\text{gear ratio} \times 16384}{360}$$

Gear ratio	Counts per joint degree	Joints
13.5:1	614.4	X
48.0:1	2184.5	A
67.82:1	3086.56	B, C
150.0:1	6826.7	Y, Z

Caution: Three wrong conversion constants exist in this project’s history

Do not use any of them.

1. “About 3300 counts per 90°” describes the *motor shaft*, not the joint. Through 67.82:1 that same motion is only about 1.07° of joint travel.
2. COUNTS_PER_REV = 16050. The encoder is 14-bit, so 16384 is correct and source-verified.
3. pulses_per_degree = -813.0 in `mks_driver.py`, off by roughly 3.8 times, with an accompanying “safety cage” that estimated limits using a different multiplier than the one actually transmitted, so it could not reliably block an unsafe command.

Use only the formula above.

10 Endstops and homing

10.1 Where the sensor actually connects

There are two wiring schemes and they are mutually exclusive. The vendor default puts a limit switch on EVCC, EGND and IN_1, all on the same green terminal block as the CAN pair — top to bottom, that block is 5V, GND, IN, CAN_H, CAN_L. The silkscreen is split, 5V/GND/IN printed above the block and CAN H L below it, which is easy to misread if you only look at one end.

This build does not use that scheme. It follows the Arctos documentation, which for the 42D axes enables *limit port remap* and moves the limits to the opposite connector: left limit to `En`, right limit to `Dir`, with `Com` tied to the matching high level. `En` and `Dir` are opto-isolated, so with `Com` floating they register nothing at all.

Caution: The 42D has no IN_2, which is why the remap exists

Per the vendor manual’s IO table, IN_1 (home / left limit) is present on both 57D and 42D, but IN_2 (right limit) is 57D only. Remapping is how a 42D gets a second endstop. On a 57D the remap is optional, since that board has a real IN_2.

Caution: No DIP switch is involved on the 42D

The SW1 pin-2 instruction that circulates for these boards belongs to the 57D. On the 42D, EVCC/EGND is the board’s own 5 V, rated 20 mA — the whole budget for the sensor and its indicator LEDs.

10.2 Enabling the remap

```
# LIMIT_PORT_REMAP (0x9E), enable = 01, CAN ID 0x06
# checksum 0x06 + 0x9E + 0x01 = 0xA5
cansend can0 006#9E01A5
# reply 9E 01 A5 -> status 1, set success
```

The manual documents 0x9E as a write only. **Nothing reports whether remap is currently on**, so it cannot be queried — only set. Two further conditions must hold or it does nothing useful:

the driver must be in a serial (SR_) work mode, because in any CR_ mode En and Dir are the pulse interface's own enable and direction inputs; and Com must be wired.

10.3 Reading the pins: 0x34

The reply is [0x34] [status] [checksum]. The status byte:

Bit	Pin	After remap, reports
bit 0	IN_1	the En pin
bit 1	IN_2	the Dir pin
bit 2	OUT_1	stall indication, 57D only
bit 3	OUT_2	reserved, 57D only
bits 4–7	reserved	

Reading 0x34 does *not* require EndLimit; that setting only governs whether the driver acts on a limit. Measured on Joint C with a dual-output hall sensor, one output per magnet pole:

Byte	Meaning
0x03	both idle resting state
0x02	bit 0 low north pole on En
0x01	bit 1 low south pole on Dir

Both are **active low**, so the default Hm_Trig = Low is correct and needs no change.

Caution: A resting 0x01 before remap looks identical to a dead sensor

Without the remap, bit 0 reports the physical IN_1 terminal. On this build that terminal is deliberately empty, so it idles high forever regardless of the magnet. Two sessions and two different sensors were spent on this before the cause was found. If an endstop reads nothing, re-send 9E 01 before suspecting the wiring.

10.4 Homing

SET_HOME_PARAMS (0x90) carries homeTrig (0 low, 1 high), homeDir (0 CW, 1 CCW), a 16-bit homeSpeed in rpm, and EndLimit (0 disable, 1 enable). GO_HOME (0x91) answers 0 fail, 1 started, 2 complete. The manual requires a GoHome after enabling EndLimit or changing any limit parameter.

Caution: EndLimit=1 unlocks the shaft at the left limit

The manual is explicit: when EndLimit is enabled and a GoHome reaches the left limit switch, the driver **unlocks the shaft**. On a belted, gravity-loaded joint that is a drop. Test with the belt off. Set EndLimit=0 if the axis must stay locked after homing.

11 Safety

Action	Frame	Example, Joint A at 0x04
Enable coils	F3 01 crc	cansend can0 004#F301F8
Disable coils	F3 00 crc	cansend can0 004#F300F7
Emergency stop	F7 crc	cansend can0 004#F7FB

Recompute for the joint you are actually driving, or take the row from BASIC_CAN_COMMANDS.

Caution: Compute your joint's e-stop frame before you need it

A previous version of this documentation gave a joint's emergency stop as `cansend can0 001#F7F8`, addressed to CAN ID 0x01, when the motor was not on that ID. It would have done nothing. Derive the frame for the joint in front of you, and keep it pasted in a second terminal before commanding motion.

Caution: Dropping coils on a gravity-loaded joint lets it fall

Several scripts in this package disable the motor in a `finally` block. On an unloaded bench motor that is harmless; on an assembled arm it removes holding torque. Know which you have before running one.

12 Troubleshooting

Symptom	Likely cause and fix
can0 does not exist	<code>slcand</code> died, usually because the adapter re-enumerated. Re-run the bring-up. Check <code>/dev/serial/by-id/</code> for the current device name.
Adapter present, can0 missing after a replug	Expected. <code>slcand</code> does not restart itself. A udev rule keyed on the adapter's serial can automate this.
<code>ip</code> shows <code>bitrate 0</code>	Normal on <code>slcan</code> . Not a fault.
No response from any ID	Check panel CAN ID, CAN rate and CANRSP; check CAN_H, CAN_L and termination; confirm the driver is powered. If still silent see Section 12.1.
Command acknowledged, motor does not move	Distinguish “not being driven” from “driven but cannot move the load” — the bus cannot tell these apart. Disable coils and turn the joint by hand.
Motion for a few degrees, then nothing in either direction	If it cannot even back off, suspect a bind stiff enough to absorb full torque, a marginal motor phase connection, or a misassembled gear. Reseating a connector once took a joint's travel budget from 1.05° to 3.85°.
Stops at the same angle every attempt	Mechanical, not roughness. Open the gearbox. A gear in the wrong position produced exactly this on Joint B.

Symptom	Likely cause and fix
Gritty resistance by hand, stalls under power	Gear mesh needs lubrication. Use a plastic-safe PTFE or silicone grease. Petroleum jelly works as a diagnostic but migrates once warm.
Motor warm but not hot	Normal. Thermal shutdown makes a motor too hot to touch; warm is a stepper doing its job.
Driver stops responding mid-test	Check power and connectors first. This has been an accidental unplug more than once.

12.1 Total silence with a powered driver: the RS485 variant

The MKS SERVO42D and SERVO57D ship in both CAN and RS485 variants. The RS485 boards have **no CAN transceiver at all** and will never appear on the bus at any bitrate or address. Joint X's driver turned out to be the RS485 variant, which is why it has never communicated. If a joint is silent across a full ID scan and a bitrate sweep, with power and wiring confirmed, **check the board's part number before spending any more time in software**.

13 The scripts in this package

All of these live in `~/arctos_ws/src/arctos_can_control/arctos_can_control`. Each takes `-can-id` and `-gear-ratio`; the motion ones also take an explicit safety band.

Script	What it does
<code>can_id_scan.py</code>	Read-only sweep of CAN IDs using 0x31. Never enables coils. Run this before trusting any ID.
<code>joint_reliability_test.py</code>	Statistical encoder sampling at rest. Response rate, checksum failures, jitter, drift.
<code>joint_unstick_move_test.py</code>	Stepped 0xFD travel in one direction with a back-off-and-retry cycle for a gritty mesh.
<code>joint_sweep_unstick_test.py</code>	Sweeps back and forth between two bounds for a fixed wall-clock duration, with the same unstick cycle and a total stall budget as a heat guard. Use when a rough spot needs crossing repeatedly.
<code>joint_speed_limit_test.py</code>	Ladder of commanded speeds across a fixed arc. Reports steady rpm, a whole-move average as a cross-check, and delivered position against commanded.
<code>joint_position_control_test.py</code>	Stepped position moves with retry. Superseded by the unstick variants for a suspect mesh.
<code>joint_endstop_test.py</code>	Read-only live monitor of the 0x34 byte. Sends nothing else, so it is safe on an assembled arm. Prints only when the byte changes.
<code>joint_endstop_dwell.py</code>	Timestamps every endstop edge and reports how long each trigger held, with a solid/chatter/marginal verdict. Use it when the monitor above shows a trigger but you need to know whether it is stable; that script has no timestamps, so a 50 ms tap and a two-second hold look identical there.

Script	What it does
<code>joint_io_diff.py</code>	Sweeps every documented read-only register with the magnet away and then held on, reporting any byte that differs. For when an endstop does not appear on 0x34 at all.
<code>joint_magnet_probe.py</code>	Checks whether an external magnet disturbs the encoder reading.

14 ROS 2 package layout

The workspace is a standard ROS 2 layout. The CAN work lives in `src/arctos_can_control`; `src/ros2_arctos` carries the description, MoveIt configuration and hardware interface.

```
cd ~/arctos_ws
colcon build --packages-select arctos_can_control
source install/setup.bash
```

The Python test scripts above do not need the workspace built or sourced. They talk to SocketCAN directly and can be run from the source directory, which is why every procedure in the SOPs uses them rather than ROS 2 nodes.

15 Corrections, and what this set replaces

These four documents replace five older ones. The following items were **wrong** in those files and were corrected here, so do not re-derive them from any surviving copy.

- **Joint B and C CAN IDs.** Joint B is 0x05 and Joint C is 0x06, as the older files and the controller config state. An interim revision of this document asserted the reverse after a scan found Joint B answering on 0x06; that driver's panel ID was wrong, and the map was right. Scan before driving, and correct the panel.
- **Encoder opcode.** An old guide standardized on 0x30 and described 0x31 as unverified. That is backwards. 0x31 is what the real C++ driver uses and what every script here uses.
- **Bitrate verification.** An old guide instructed the reader to run `ip link set can0 up type can bitrate 500000` and then verify by looking for `bitrate 500000`. Neither works on `slcan`.
- **Bus error counters.** Old troubleshooting advice leaned on bus-error counts that the `slcan` driver never populates.
- **Motion command.** An old safe-first-motion procedure was built on 0xF5, whose absolute-versus-relative behaviour it correctly flagged as unresolved. That question is moot: use 0xFD.
- **Missing checksums.** The old cheat sheet listed `cansend can0 005#F300` and `cansend can0 005#F7` with no checksum byte at all. Both would be rejected. The correct frames are 005#F300F8 and 005#F7FC.
- **Speed ceiling.** The no-load figure of 225 rpm was presented as a general operating limit. It is a bare-motor number; Joint B saturates at about 165.